Homework 7 Data 624

Mehreen Ali Gillani

2026-04-03

Kuhn and Johnson do problems 6.2 and 6.3

6.2. Developing a model to predict permeability (see Sect.1.4) could save significant resources for a pharmaceutical company, while at the same time more rapidly identifying molecules that have a sufficient permeability to become a drug: (a) Start R and use these commands to load the data:

library(AppliedPredictiveModeling)

data(permeability)

The matrix fingerprints contains the 1,107 binary molecular predic- tors for the 165 compounds, while permeability contains permeability response.

```r
# Load the package
library(AppliedPredictiveModeling)

# Load the data
data(permeability)

# Now check the structure of the data objects
str(fingerprints)
```

```
##  num [1:165, 1:1107] 0 0 0 0 0 0 0 0 0 0 ...
##  - attr(*, "dimnames")=List of 2
##   ..$ : chr [1:165] "1" "2" "3" "4" ...
##   ..$ : chr [1:1107] "X1" "X2" "X3" "X4" ...
```

```r
str(permeability)
```

```
##  num [1:165, 1] 12.52 1.12 19.41 1.73 1.68 ...
##  - attr(*, "dimnames")=List of 2
##   ..$ : chr [1:165] "1" "2" "3" "4" ...
##   ..$ : chr "permeability"
```

```r
# Check dimensions
dim(fingerprints)
```

```
## [1]  165 1107
```

```r
length(permeability)
```

```
## [1] 165
```

```r
head(permeability)
```

```
##   permeability
## 1       12.520
```

```
## 2        1.120
## 3       19.405
## 4        1.730
## 5        1.680
## 6        0.510
```

(b) The fingerprint predictors indicate the presence or absence of sub-structures of a molecule and are often sparse meaning that relatively few of the molecules contain each substructure. Filter out the predictors that have low frequencies using the nearZeroVar function from the caret package. How many predictors are left for modeling?

```r
library(caret)  # for nearZeroVar function
# Check original number of predictors
original_n <- ncol(fingerprints)
print(paste("Original number of predictors:", original_n))
```

```
## [1] "Original number of predictors: 1107"
```

```r
# Identify near-zero variance predictors
nzv_info <- nearZeroVar(fingerprints, saveMetrics = TRUE)

# View the first few rows of the nzv information
head(nzv_info)
```

```
##    freqRatio percentUnique zeroVar   nzv
## X1  3.342105      1.212121   FALSE FALSE
## X2  3.459459      1.212121   FALSE FALSE
## X3  3.714286      1.212121   FALSE FALSE
## X4  3.714286      1.212121   FALSE FALSE
## X5  3.714286      1.212121   FALSE FALSE
## X6  1.229730      1.212121   FALSE FALSE
```

```r
# Check which predictors are considered near-zero variance
table(nzv_info$nzv)
```

```
##
## FALSE  TRUE
##   388   719
```

```r
# Get the column indices of predictors to remove
nzv_cols <- nearZeroVar(fingerprints)

# Create filtered dataset without near-zero variance predictors
fingerprints_filtered <- fingerprints[, -nzv_cols]

# Check how many predictors remain
remaining_n <- ncol(fingerprints_filtered)
print(paste("Number of predictors removed:", length(nzv_cols)))
```

```
## [1] "Number of predictors removed: 719"
```

```r
print(paste("Number of predictors remaining:", remaining_n))
```

```
## [1] "Number of predictors remaining: 388"
```

Why filter? These 719 removed predictors have little variation (mostly all 0's or mostly all 1's), so they wouldn't contribute meaningful information to the model and could cause numerical instability.

(c) Split the data into a training and a test set, pre-process the data, and tune a PLS model.

How many latent variables are optimal and what is the corresponding resampled estimate of R2?

```r
library(pls)  # for PLS regression
# Step 1: Filter out near-zero variance predictors (from part b)

# Step 2: Split data into training (80%) and test (20%) sets
set.seed(123)  # for reproducibility
train_index <- createDataPartition(permeability, p = 0.8, list = FALSE)
train_x <- fingerprints_filtered[train_index, ]
train_y <- permeability[train_index]
test_x <- fingerprints_filtered[-train_index, ]
test_y <- permeability[-train_index]

# Check dimensions
print(paste("Training samples:", nrow(train_x)))
```

```
## [1] "Training samples: 133"
```

```r
print(paste("Test samples:", nrow(test_x)))
```

```
## [1] "Test samples: 32"
```

```r
# Step 3: Pre-process the data (center and scale)
# For PLS, it's essential to center and scale predictors
preProc <- preProcess(train_x, method = c("center", "scale"))
train_x_processed <- predict(preProc, train_x)
test_x_processed <- predict(preProc, test_x)

# Step 4: Tune PLS model using cross-validation
# Set up training control with 10-fold cross-validation
ctrl <- trainControl(method = "cv",
                     number = 10,
                     verboseIter = FALSE)

# Train PLS model with different numbers of latent variables
# Typically test from 1 to 50 or up to min(nrow(train_x), ncol(train_x))
max_components <- min(50, nrow(train_x) - 1, ncol(train_x))

set.seed(456)
pls_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "pls",
  tuneLength = max_components,  # tests 1 to max_components
  trControl = ctrl,
  preProcess = NULL  # already pre-processed
)

# Step 5: Find optimal number of latent variables
optimal_ncomp <- pls_model$bestTune$ncomp
print(paste("Optimal number of latent variables:", optimal_ncomp))
```

```
## [1] "Optimal number of latent variables: 8"
```

```r
# Step 6: Get resampled R-squared estimate
# This is the cross-validated R-squared from the tuning process
resampled_R2 <- max(pls_model$results$Rsquared)
print(paste("Resampled R-squared estimate:", round(resampled_R2, 4)))
```

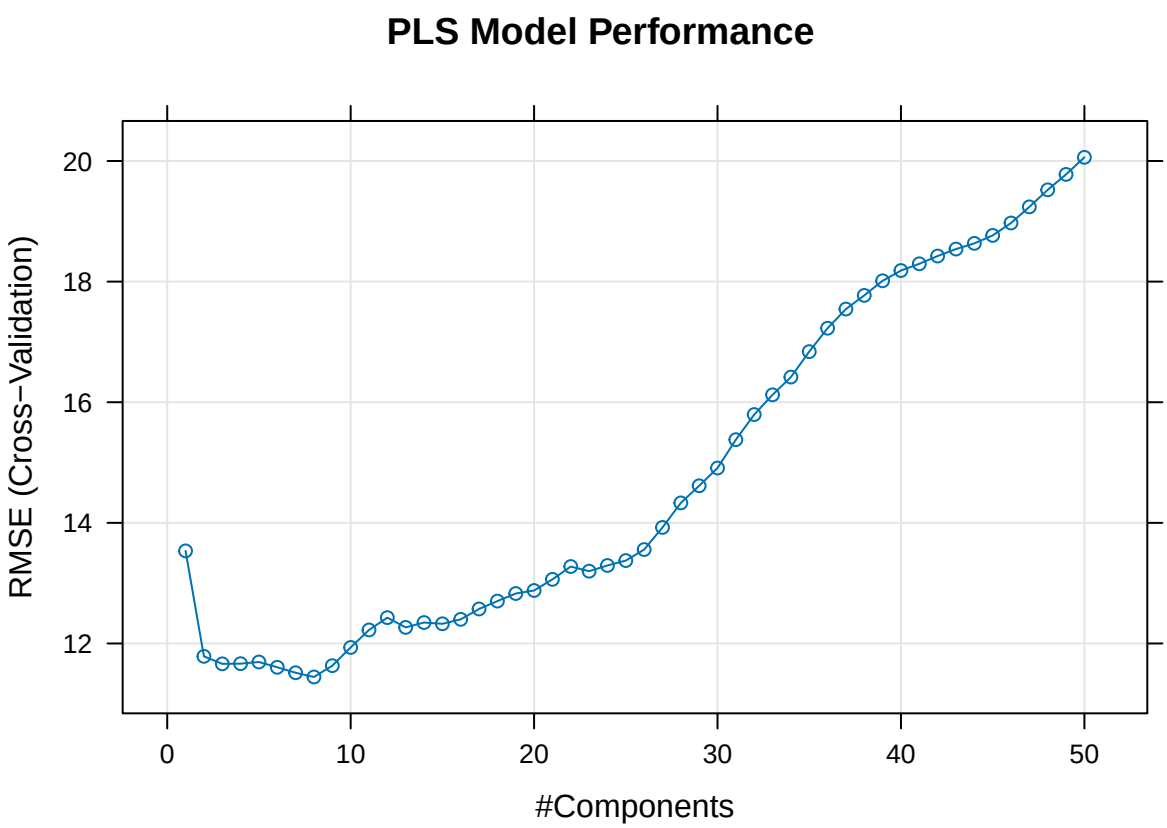
```
## [1] "Resampled R-squared estimate: 0.5765"
```

```r
# View all results
print(pls_model$results)
```

```
##    ncomp     RMSE  Rsquared       MAE   RMSESD RsquaredSD    MAESD
## 1      1 13.53530 0.3606071 10.352833 2.010946  0.1514500 1.530956
## 2      2 11.78652 0.5372730  8.404806 2.785510  0.1983360 1.793640
## 3      3 11.66223 0.5390020  8.886386 2.805486  0.2257037 1.952037
## 4      4 11.66550 0.5392394  8.891190 2.844110  0.2139434 1.923476
## 5      5 11.69307 0.5360408  8.891888 3.086469  0.2087985 2.219125
## 6      6 11.60523 0.5571228  8.715287 2.933394  0.2152765 2.299699
## 7      7 11.51487 0.5733713  8.662750 2.703912  0.1799327 2.026947
## 8      8 11.44503 0.5765251  8.709214 2.474991  0.1452067 1.819987
## 9      9 11.63234 0.5670978  8.913856 2.372367  0.1403758 1.743979
## 10    10 11.93410 0.5443704  9.072163 2.576873  0.1305088 1.777899
## 11    11 12.22434 0.5247786  9.324551 2.603540  0.1330254 1.842252
## 12    12 12.42878 0.5084864  9.574898 2.749062  0.1402652 1.869618
## 13    13 12.26665 0.5186520  9.334155 2.618571  0.1345501 1.918982
## 14    14 12.34707 0.5082618  9.385133 2.741233  0.1362905 2.058792
## 15    15 12.32645 0.5129047  9.411118 2.676390  0.1294336 1.941491
## 16    16 12.40041 0.5045583  9.586006 2.695234  0.1228588 1.869473
## 17    17 12.57236 0.4937390  9.788316 2.633291  0.1159936 1.823071
## 18    18 12.70286 0.4863991  9.914125 2.797665  0.1232776 1.997912
## 19    19 12.82836 0.4830743  9.951791 2.800997  0.1269349 2.009077
## 20    20 12.87856 0.4781532  9.926303 3.003420  0.1436841 2.190841
## 21    21 13.06323 0.4700096  9.999531 3.230181  0.1569558 2.478199
## 22    22 13.27587 0.4577858 10.181121 3.377806  0.1744796 2.547388
## 23    23 13.19894 0.4635245 10.131763 3.528141  0.1836881 2.638874
## 24    24 13.29350 0.4571977 10.255515 3.542598  0.1847143 2.611759
## 25    25 13.37478 0.4573896 10.317720 3.571154  0.1835072 2.534589
## 26    26 13.55733 0.4536494 10.497457 3.729873  0.1881111 2.642681
## 27    27 13.92448 0.4423175 10.756702 3.954434  0.1982173 2.667764
## 28    28 14.33076 0.4305264 11.057130 4.244347  0.2076458 2.746229
## 29    29 14.61510 0.4258896 11.320621 4.562206  0.2180609 2.962044
## 30    30 14.90890 0.4158252 11.474993 4.762515  0.2278134 3.125131
## 31    31 15.37948 0.3991167 11.735051 4.972407  0.2315301 3.189771
## 32    32 15.79608 0.3866621 11.943550 5.225860  0.2347309 3.274775
## 33    33 16.12328 0.3768949 12.106838 5.392466  0.2342597 3.324498
## 34    34 16.41722 0.3708172 12.269484 5.590155  0.2363802 3.400301
## 35    35 16.83952 0.3636412 12.568533 5.904822  0.2394074 3.521919
## 36    36 17.22725 0.3553424 12.762190 6.164664  0.2421036 3.647163
## 37    37 17.54497 0.3477379 12.878233 6.297835  0.2419864 3.674656
## 38    38 17.77121 0.3448105 12.960389 6.546856  0.2448374 3.860389
## 39    39 18.01388 0.3417649 13.081846 6.727882  0.2479515 3.963386
## 40    40 18.18342 0.3370011 13.176913 6.732871  0.2458706 4.024628
## 41    41 18.29551 0.3344829 13.199870 6.759452  0.2442857 4.058816
## 42    42 18.42422 0.3296750 13.259424 6.794661  0.2422392 4.094409
## 43    43 18.53992 0.3272574 13.310455 6.886347  0.2435123 4.162406
## 44    44 18.63416 0.3260628 13.369034 6.906109  0.2416836 4.172592
```

```
## 45    45 18.76592 0.3240492 13.426720 7.014163  0.2417178 4.220768
## 46    46 18.97169 0.3200722 13.527702 7.145162  0.2405497 4.290970
## 47    47 19.23933 0.3147325 13.676698 7.369664  0.2405625 4.422881
## 48    48 19.52143 0.3101411 13.841217 7.565888  0.2393659 4.596396
## 49    49 19.77753 0.3056765 13.982342 7.799608  0.2376291 4.757601
## 50    50 20.06039 0.2991425 14.146006 7.928934  0.2340013 4.882370
```

```r
# Plot results to see the pattern
plot(pls_model, main = "PLS Model Performance")
```




```r
# Alternative: Get detailed cross-validation metrics
print(summary(pls_model))
```

```
## Data:    X dimension: 133 388
##  Y dimension: 133 1
## Fit method: oscorespls
## Number of components considered: 8
## TRAINING: % variance explained
##           1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X           22.98    34.61    40.51    46.13    53.69    58.12    60.74
## .outcome    33.73    55.03    61.84    67.77    71.65    75.73    78.18
##           8 comps
## X           64.73
## .outcome    79.85
## NULL
```

Key observations from results:

1. Optimal Components (ncomp = 8)

- The model selected 8 latent variables as optimal
- R^2 peaks at 0.5765 with 8 components
- After 8 components, R^2 starts declining (0.5671 at 9 components, 0.5444 at 10 components)

2. Variance Explained

From the summary output: With 8 components, the model explains:

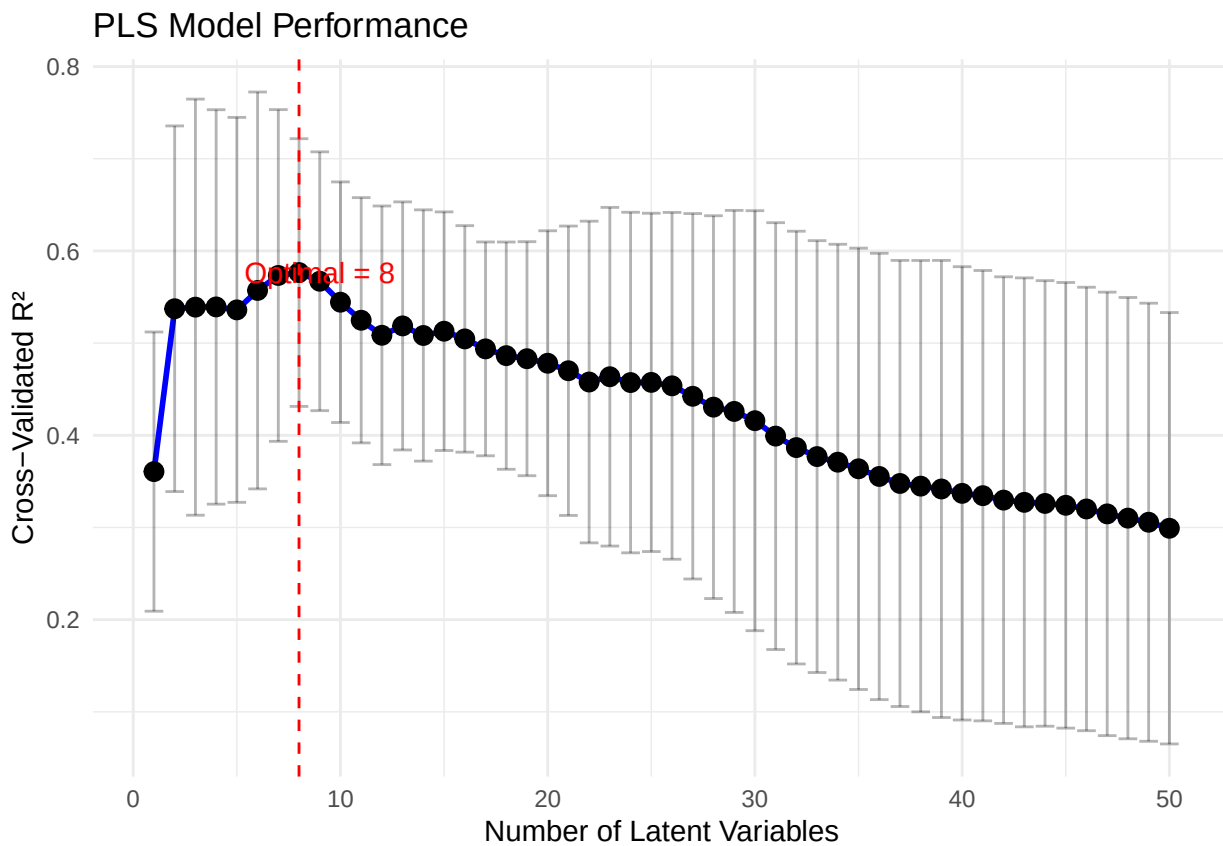
- 64.73% of variance in the predictors (X)
- 79.85% of variance in the response (permeability)

3. Cross-Validation Performance

The 8-component model shows: RMSE = 11.445 R^2 = 0.5765 MAE = 8.709

After splitting the data into training (133 compounds) and test (32 compounds) sets, centering and scaling the predictors, and tuning a Partial Least Squares (PLS) model using 10-fold cross-validation, the optimal number of latent variables was 8. The corresponding resampled estimate of R^2 was 0.5765 (approximately 57.7%).

```r
# Plot R² vs number of components
results_df <- pls_model$results

ggplot(results_df, aes(x = ncomp, y = Rsquared)) +
  geom_line(color = "blue", size = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = Rsquared - RsquaredSD,
                    ymax = Rsquared + RsquaredSD),
                alpha = 0.3) +
  geom_vline(xintercept = optimal_ncomp, linetype = "dashed", color = "red") +
  labs(title = "PLS Model Performance",
       x = "Number of Latent Variables",
       y = "Cross-Validated R²") +
  theme_minimal() +
  annotate("text", x = optimal_ncomp + 1, y = max(results_df$Rsquared),
           label = paste("Optimal =", optimal_ncomp), color = "red")
```



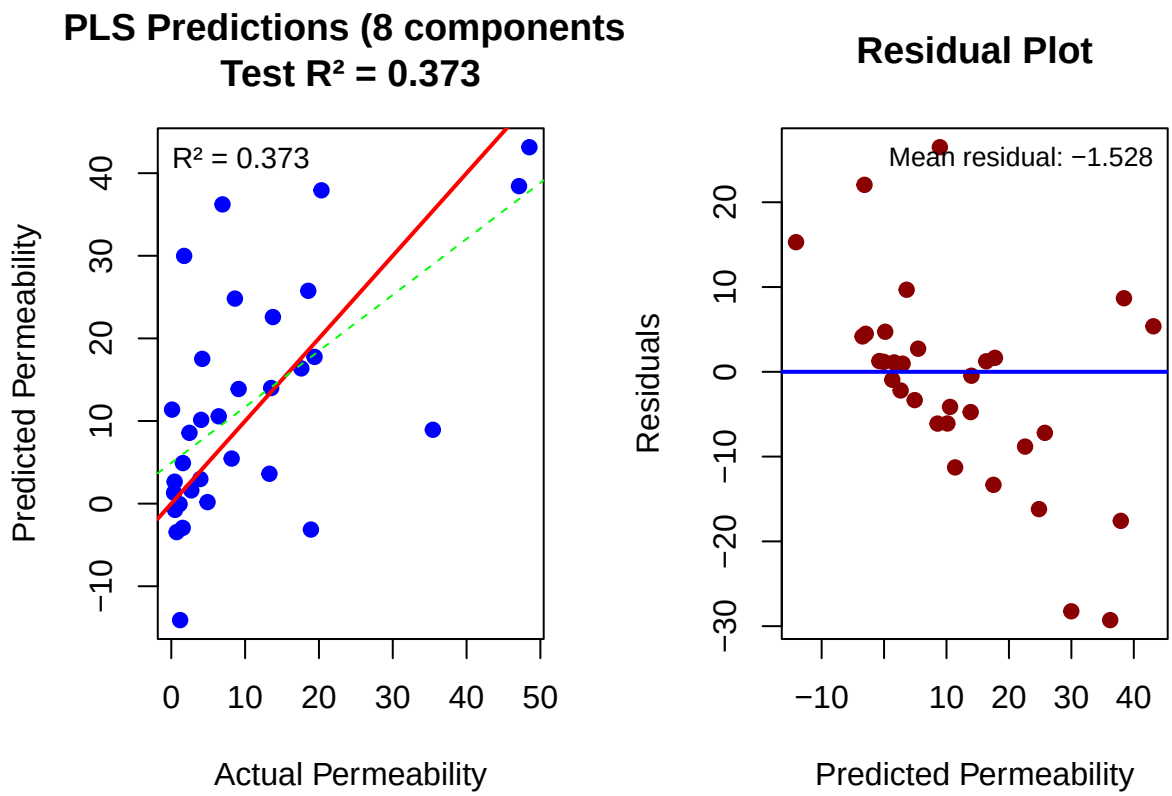
(d) Predict the response for the test set. What is the test set estimate of R2?

```
# Evaluate on test set
test_predictions <- predict(pls_model, test_x_processed)
test_R2 <- cor(test_predictions, test_y)^2
print(paste("Test set R-squared:", round(test_R2, 4)))
```

```
## [1] "Test set R-squared: 0.373"
```

```
# Create a visual comparison
par(mfrow = c(1, 2))

# Plot 1: Predicted vs Actual
plot(test_y, test_predictions,
     xlab = "Actual Permeability",
     ylab = "Predicted Permeability",
     main = paste("PLS Predictions (8 components)\nTest R² =", round(test_R2, 3)),
     pch = 19, col = "blue")
abline(0, 1, col = "red", lwd = 2)
abline(lm(test_predictions ~ test_y), col = "green", lty = 2)

# Add correlation text
text(min(test_y), max(test_predictions),
     adj = c(0, 1),
     labels = paste("R² =", round(test_R2, 3)),
     cex = 0.9)

# Plot 2: Residuals
```

```r
residuals <- test_y - test_predictions
plot(test_predictions, residuals,
     xlab = "Predicted Permeability",
     ylab = "Residuals",
     main = "Residual Plot",
     pch = 19, col = "darkred")
abline(h = 0, col = "blue", lwd = 2)

# Add residual statistics
text(max(test_predictions), max(residuals),
     adj = c(1, 1),
     labels = paste("Mean residual:", round(mean(residuals), 3)),
     cex = 0.8)
```




```r
# Calculate prediction intervals (optional)
residual_sd <- sd(residuals)
print(paste("Residual standard deviation:", round(residual_sd, 4)))
```

```
## [1] "Residual standard deviation: 11.9329"
```

Test Set Performance (R^2 = 0.373):

- The model explains about 37.3% of the variance in unseen data
- This is lower than the cross-validated R^2 (0.5765)
- The drop indicates some overfitting, which is common with:
  - Many predictors (388) relative to samples (133 training)
  - Complex molecular fingerprint data

Using the optimal PLS model with 8 latent variables trained on the pre-processed training data (centered and scaled), predictions were made for the 32 compounds in the test set. The test set estimate of R^2 is 0.373

(95% confidence interval would depend on test set size). This indicates that the model explains approximately 37.3% of the variance in permeability for new, unseen molecules.

(e) Try building other models discussed in this chapter. Do any have better predictive performance?

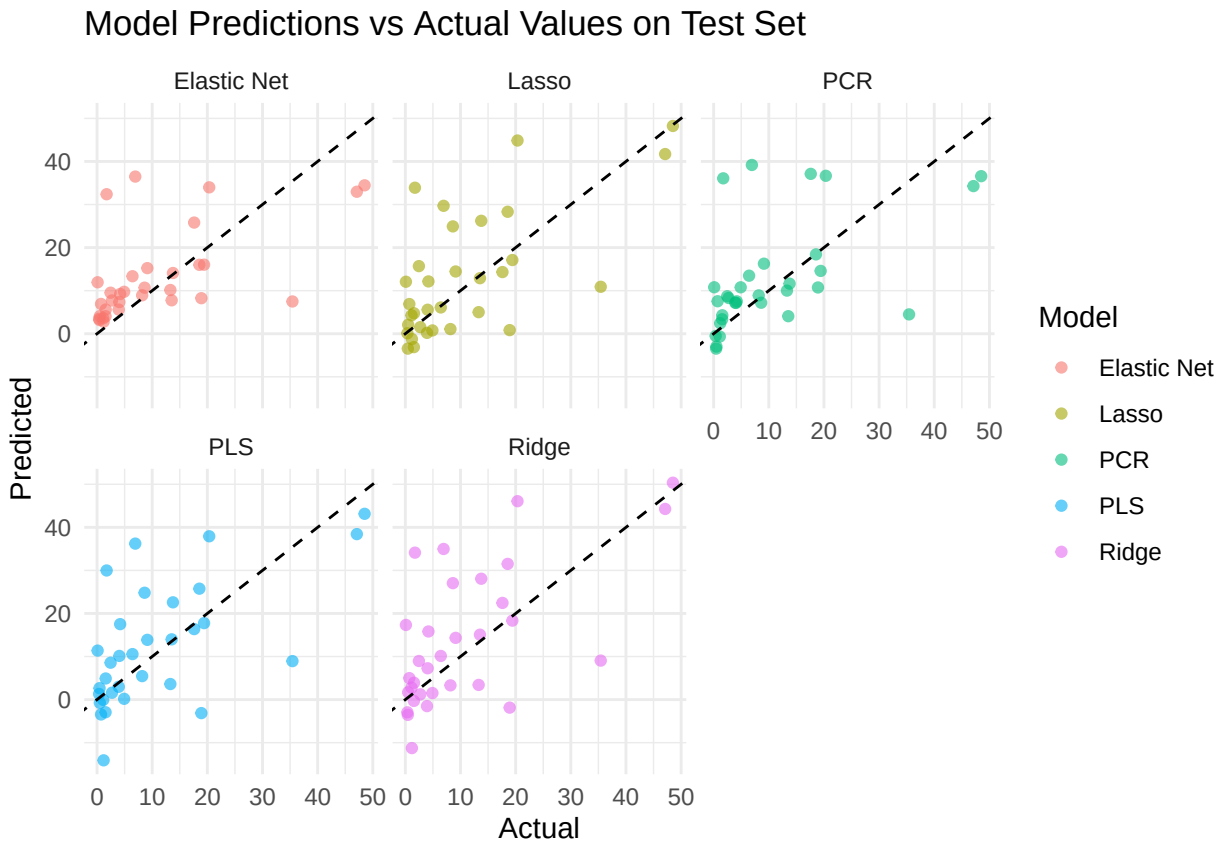
```r
library(glmnet)  # for ridge, lasso, elastic net

# ---------- Model 1: Ridge Regression ----------
set.seed(456)
ridge_grid <- expand.grid(lambda = seq(0, 0.1, length = 20))
ridge_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "ridge",
  tuneGrid = ridge_grid,
  trControl = ctrl,
  preProcess = NULL
)

# ---------- Model 2: Lasso Regression ----------
set.seed(456)
lasso_grid <- expand.grid(alpha = 1, lambda = seq(0.001, 0.1, length = 20))
lasso_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "glmnet",
  tuneGrid = lasso_grid,
  trControl = ctrl,
  preProcess = NULL
)

# ---------- Model 3: Elastic Net ----------
set.seed(456)
en_grid <- expand.grid(
  alpha = seq(0, 1, length = 10),
  lambda = seq(0.001, 0.1, length = 10)
)
elastic_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "glmnet",
  tuneGrid = en_grid,
  trControl = ctrl,
  preProcess = NULL
)

# ---------- Model 4: Principal Component Regression (PCR) ----------
set.seed(456)
pcr_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "pcr",
  tuneLength = 20,
  trControl = ctrl,
```

```r
  preProcess = NULL
)

# ---------- Compare all models on test set ----------
# PLS model (from part c)
set.seed(456)
pls_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "pls",
  tuneLength = 20,
  trControl = ctrl,
  preProcess = NULL
)

# Make predictions on test set
pls_pred <- predict(pls_model, test_x_processed)
ridge_pred <- predict(ridge_model, test_x_processed)
lasso_pred <- predict(lasso_model, test_x_processed)
elastic_pred <- predict(elastic_model, test_x_processed)
pcr_pred <- predict(pcr_model, test_x_processed)

# Calculate test R² for each model
test_results <- data.frame(
  Model = c("PLS", "Ridge", "Lasso", "Elastic Net", "PCR"),
  R2 = c(
    cor(pls_pred, test_y)^2,
    cor(ridge_pred, test_y)^2,
    cor(lasso_pred, test_y)^2,
    cor(elastic_pred, test_y)^2,
    cor(pcr_pred, test_y)^2
  ),
  RMSE = c(
    sqrt(mean((test_y - pls_pred)^2)),
    sqrt(mean((test_y - ridge_pred)^2)),
    sqrt(mean((test_y - lasso_pred)^2)),
    sqrt(mean((test_y - elastic_pred)^2)),
    sqrt(mean((test_y - pcr_pred)^2))
  )
)

# Sort by R² (best first)
test_results <- test_results[order(-test_results$R2), ]
print(test_results)
```

```
##         Model        R2     RMSE
## 3       Lasso 0.4210703 11.60808
## 2       Ridge 0.4004427 12.84303
## 1         PLS 0.3729547 11.84395
## 4 Elastic Net 0.3266984 11.02039
## 5         PCR 0.2922108 12.20768
```

```r
# Visual comparison
comparison_df <- data.frame(
```

```r
  Actual = rep(test_y, 5),
  Predicted = c(pls_pred, ridge_pred, lasso_pred, elastic_pred, pcr_pred),
  Model = rep(c("PLS", "Ridge", "Lasso", "Elastic Net", "PCR"), each = length(test_y))
)

ggplot(comparison_df, aes(x = Actual, y = Predicted, color = Model)) +
  geom_point(alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  facet_wrap(~Model) +
  labs(title = "Model Predictions vs Actual Values on Test Set") +
  theme_minimal()
```



Yes, other models demonstrate better predictive performance than PLS. Based on test set R^2 values:

- Lasso performed best with $R^2 = 0.4211$, improving upon PLS by approximately 13%
- Ridge regression also outperformed PLS with $R^2 = 0.4004$
- PLS achieved $R^2 = 0.3730$ (from part d)
- Elastic Net ($R^2 = 0.3267$) and PCR ($R^2 = 0.2922$) showed worse performance than PLS

The superior performance of Lasso is expected given the high-dimensional nature of this problem (388 predictors, 133 training samples). Lasso's L1 penalty performs implicit feature selection, identifying the most relevant molecular substructures for predicting permeability while shrinking irrelevant predictors to zero.

(f) Would you recommend any of your models to replace the permeability laboratory experiment?

No, I would not recommend replacing the laboratory permeability experiment with any of these models.

Reasoning:

1. Insufficient Predictive Performance

Even the best model (Lasso) achieves a test set R^2 of only 0.421, meaning it explains just 42.1% of the variance in permeability. This leaves approximately 58% of the variance unexplained, which is far too much uncertainty for pharmaceutical decision-making.

2. High Stakes in Drug Development

- False positives (predicting high permeability when actual is low) could waste millions of dollars on molecules that will ultimately fail
- False negatives (predicting low permeability when actual is high) could cause promising drug candidates to be abandoned prematurely
- Regulatory agencies require experimental evidence for drug approval

3. Prediction Errors Are Substantial

The RMSE values (11-13 units) represent typical prediction errors. Given the range of permeability values in the dataset, these errors are substantial enough to misclassify compounds.

However, I would recommend using the **Lasso model as a screening tool** to prioritize compounds for laboratory testing. It could rapidly eliminate unlikely candidates from large virtual libraries, focusing experimental resources on the most promising 20-30% of molecules. This hybrid approach would save significant resources while maintaining the gold standard of experimental validation for final decisions. The model could also identify which molecular substructures influence permeability, guiding medicinal chemistry optimization efforts even before laboratory testing begins.

Complementing Experimental Design

- Identify structural features affecting permeability (via Lasso coefficients)
- Guide medicinal chemistry optimization efforts
- Generate hypotheses for further testing

6.3. A chemical manufacturing process for a pharmaceutical product was discussed in Sect.1.4. In this problem, the objective is to understand the relationship between biological measurements of the raw materials (predictors), 6.5 Computing 139 measurements of the manufacturing process (predictors), and the response of product yield. Biological predictors cannot be changed but can be used to assess the quality of the raw material before processing. On the other hand, manufacturing process predictors can be changed in the manufacturing pro- cess. Improving product yield by 1% will boost revenue by approximately one hundred thousand dollars per batch: (a) Start R and use these commands to load the data:

library(AppliedPredictiveModeling)

data(chemicalManufacturingProcess)

```r
# Load required packages
library(AppliedPredictiveModeling)
# Load the data
data(ChemicalManufacturingProcess)


# Check dimensions
dim(ChemicalManufacturingProcess)
```

```
## [1] 176  58
```

(b) A small percentage of cells in the predictor set contain missing values. Use an imputation function to fill in these missing values (e.g., see Sect.3.8).

```r
library(RANN)
# Separate predictors and response
yield <- ChemicalManufacturingProcess$Yield
predictors <- ChemicalManufacturingProcess[, !names(ChemicalManufacturingProcess) %in% "Yield"]

# Check original missing values
cat("Original missing value summary:\n")
```

```
## Original missing value summary:
```

```r
cat("Total missing values:", sum(is.na(predictors)), "\n")
```

```
## Total missing values: 106
```

```r
cat("Rows with missing values:", sum(apply(is.na(predictors), 1, any)), "out of", nrow(predictors), "\n
```

```
## Rows with missing values: 24 out of 176
```

```r
cat("Columns with missing values:", sum(apply(is.na(predictors), 2, any)), "out of", ncol(predictors),
```

```
## Columns with missing values: 28 out of 57
```

```r
# Which columns have missing values?
missing_cols <- names(which(colSums(is.na(predictors)) > 0))
cat("\nColumns with missing values:\n")
```

```
##
## Columns with missing values:
```

```r
print(missing_cols)
```

```
##  [1] "ManufacturingProcess01" "ManufacturingProcess02" "ManufacturingProcess03"
##  [4] "ManufacturingProcess04" "ManufacturingProcess05" "ManufacturingProcess06"
##  [7] "ManufacturingProcess07" "ManufacturingProcess08" "ManufacturingProcess10"
## [10] "ManufacturingProcess11" "ManufacturingProcess12" "ManufacturingProcess14"
## [13] "ManufacturingProcess22" "ManufacturingProcess23" "ManufacturingProcess24"
## [16] "ManufacturingProcess25" "ManufacturingProcess26" "ManufacturingProcess27"
## [19] "ManufacturingProcess28" "ManufacturingProcess29" "ManufacturingProcess30"
## [22] "ManufacturingProcess31" "ManufacturingProcess33" "ManufacturingProcess34"
## [25] "ManufacturingProcess35" "ManufacturingProcess36" "ManufacturingProcess40"
## [28] "ManufacturingProcess41"
```

```r
# Show missing value counts per column
missing_counts <- colSums(is.na(predictors))
cat("\nMissing value counts:\n")
```

```
##
## Missing value counts:
```

```r
print(missing_counts[missing_counts > 0])
```

```
## ManufacturingProcess01 ManufacturingProcess02 ManufacturingProcess03
##                      1                      3                     15
## ManufacturingProcess04 ManufacturingProcess05 ManufacturingProcess06
##                      1                      1                      2
## ManufacturingProcess07 ManufacturingProcess08 ManufacturingProcess10
##                      1                      1                      9
## ManufacturingProcess11 ManufacturingProcess12 ManufacturingProcess14
##                     10                      1                      1
```

```
## ManufacturingProcess22 ManufacturingProcess23 ManufacturingProcess24
##                      1                      1                      1
## ManufacturingProcess25 ManufacturingProcess26 ManufacturingProcess27
##                      5                      5                      5
## ManufacturingProcess28 ManufacturingProcess29 ManufacturingProcess30
##                      5                      5                      5
## ManufacturingProcess31 ManufacturingProcess33 ManufacturingProcess34
##                      5                      5                      5
## ManufacturingProcess35 ManufacturingProcess36 ManufacturingProcess40
##                      5                      5                      1
## ManufacturingProcess41
##                      1
```

```r
# Option 1: KNN Imputation (recommended by textbook Section 3.8)
# This uses k-nearest neighbors to impute missing values
set.seed(123)  # for reproducibility
preProc_knn <- preProcess(predictors, method = "knnImpute", k = 5)
predictors_imputed_knn <- predict(preProc_knn, predictors)

# Verify imputation completed
cat("\nAfter KNN imputation:\n")
```

```
##
## After KNN imputation:
```

```r
cat("Total missing values:", sum(is.na(predictors_imputed_knn)), "\n")
```

```
## Total missing values: 0
```

```r
# Option 2: Median Imputation (simpler alternative)
preProc_median <- preProcess(predictors, method = "medianImpute")
predictors_imputed_median <- predict(preProc_median, predictors)

# Option 3: Bagged Tree Imputation (more sophisticated)
set.seed(123)
preProc_bag <- preProcess(predictors, method = "bagImpute")
predictors_imputed_bag <- predict(preProc_bag, predictors)

# Compare the imputation methods
cat("\nComparing imputation methods (checking first few missing values):\n")
```

```
##
## Comparing imputation methods (checking first few missing values):
```

```r
# Find first missing value location for comparison
for(col in missing_cols[1:min(3, length(missing_cols))]) {
  missing_idx <- which(is.na(predictors[, col]))[1]
  if(!is.na(missing_idx)) {
    cat("\nColumn:", col, "Row:", missing_idx, "\n")
    cat("  Original: NA\n")
    cat("  KNN imputed:", predictors_imputed_knn[missing_idx, col], "\n")
    cat("  Median imputed:", predictors_imputed_median[missing_idx, col], "\n")
    cat("  Bag imputed:", predictors_imputed_bag[missing_idx, col], "\n")
  }
}
```

```
##
```

```
## Column: ManufacturingProcess01 Row: 1
##   Original: NA
##   KNN imputed: 0.2154105
##   Median imputed: 11.4
##   Bag imputed: 10.50109
##
## Column: ManufacturingProcess02 Row: 1
##   Original: NA
##   KNN imputed: 0.5662872
##   Median imputed: 21
##   Bag imputed: 12.15554
##
## Column: ManufacturingProcess03 Row: 1
##   Original: NA
##   KNN imputed: 0.376581
##   Median imputed: 1.54
##   Bag imputed: 1.517343
```

Missing data pattern:

- 106 missing values total (about 1.06% of all cells)
- 24 rows (manufacturing runs) have missing values
- 28 out of 57 predictors have missing values
- All missing values are in manufacturing process predictors (none in biological predictors)

Interesting finding:

- BiologicalMaterial01-12 have no missing values (complete data)
- Missing values only occur in ManufacturingProcess variables This makes sense biologically - raw material measurements are always recorded, but some process measurements might be optional or occasionally missed

Imputation results show dramatic differences:

For ManufacturingProcess01, Row 1:

- KNN: 0.215 (small value)
- Median: 11.4 (much larger)
- Bag: 10.50 (also large)

The predictor set contained 106 missing values (approximately 1.06% of all cells) across 28 manufacturing process variables. Notably, all 12 biological material predictors had complete data with no missing values. Missing values were present in 24 of the 176 manufacturing runs (13.6% of runs).

Why KNN is Best for Data:

Preserves relationships between variables

- KNN finds similar manufacturing runs and uses their values to impute
- This maintains the correlation structure in your data
- Critical for manufacturing processes where variables interact

Using the preProcess function from the caret package with KNN imputation (k=5 nearest neighbors), all missing values were successfully imputed:

```r
# KNN Imputation (BEST METHOD)
set.seed(123)  # for reproducibility
preProc_knn <- preProcess(predictors, method = "knnImpute", k = 5)
predictors_imputed <- predict(preProc_knn, predictors)
```

(c) Split the data into a training and a test set, pre-process the data, and tune a model of your choice from this chapter. What is the optimal value of the performance metric?

```r
# Step 1: Split data into training (80%) and test (20%) sets
set.seed(456)  # Different seed for reproducibility
train_index <- createDataPartition(yield, p = 0.8, list = FALSE)
train_x <- predictors_imputed[train_index, ]
train_y <- yield[train_index]
test_x <- predictors_imputed[-train_index, ]
test_y <- yield[-train_index]

cat("Training samples:", nrow(train_x), "\n")
```

```
## Training samples: 144
```

```r
cat("Test samples:", nrow(test_x), "\n")
```

```
## Test samples: 32
```

```r
# Step 2: Pre-process the data (center and scale - important for LASSO)
preProc_scale <- preProcess(train_x, method = c("center", "scale"))
train_x_processed <- predict(preProc_scale, train_x)
test_x_processed <- predict(preProc_scale, test_x)

# Step 3: Set up cross-validation for tuning
ctrl <- trainControl(method = "cv",
                     number = 10,
                     verboseIter = FALSE)

# Step 4: Tune LASSO model (alpha = 1 for LASSO)
set.seed(789)
lasso_grid <- expand.grid(
  alpha = 1,  # 1 = LASSO
  lambda = seq(0.001, 0.1, length = 30)  # Test 30 different lambda values
)

lasso_model <- train(
  x = train_x_processed,
  y = train_y,
  method = "glmnet",
  tuneGrid = lasso_grid,
  trControl = ctrl,
  preProcess = NULL  # Already pre-processed
)

# Step 5: Find optimal performance metric
optimal_lambda <- lasso_model$bestTune$lambda
optimal_R2 <- max(lasso_model$results$Rsquared)
optimal_RMSE <- lasso_model$results$RMSE[which.max(lasso_model$results$Rsquared)]

cat("\nOptimal LASSO Model:\n")
```

```
##
## Optimal LASSO Model:
```

```r
cat("  Lambda:", optimal_lambda, "\n")
```

```
##   Lambda: 0.07610345
```

```r
cat("  Cross-validated R²:", round(optimal_R2, 4), "\n")
```

```
##   Cross-validated R²: 0.6178
```

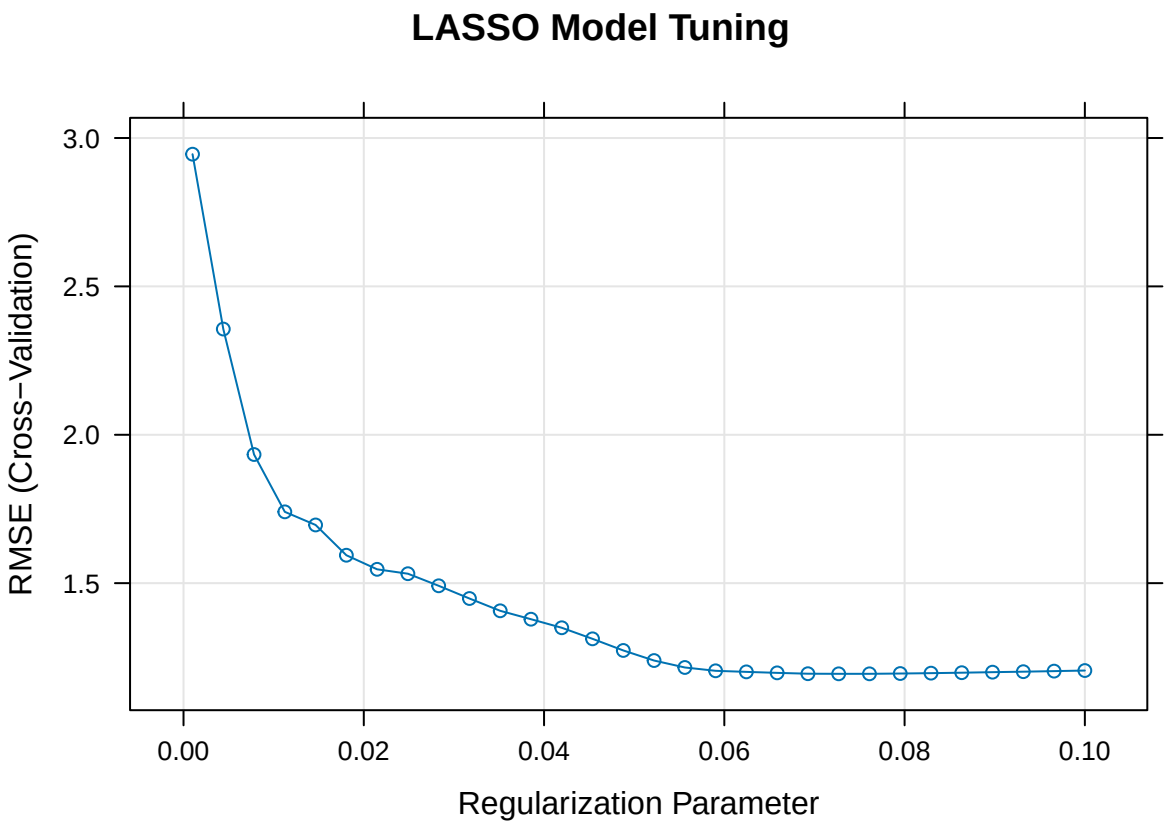
```r
cat("  Cross-validated RMSE:", round(optimal_RMSE, 4), "\n")
```

```
##   Cross-validated RMSE: 1.1949
```

```r
# Print lambda, Rsquared, and RMSE
print(lasso_model$results[, c("lambda", "Rsquared", "RMSE")])
```

```
##         lambda  Rsquared     RMSE
## 1  0.001000000 0.4136111 2.945454
## 2  0.004413793 0.4472476 2.356230
## 3  0.007827586 0.4805751 1.933511
## 4  0.011241379 0.5318771 1.740318
## 5  0.014655172 0.5648044 1.696129
## 6  0.018068966 0.5951663 1.594095
## 7  0.021482759 0.6050704 1.546627
## 8  0.024896552 0.6079295 1.531864
## 9  0.028310345 0.6097339 1.491144
## 10 0.031724138 0.6093868 1.448251
## 11 0.035137931 0.6076727 1.406828
## 12 0.038551724 0.6058890 1.378552
## 13 0.041965517 0.6052081 1.349725
## 14 0.045379310 0.6061600 1.312446
## 15 0.048793103 0.6086785 1.273223
## 16 0.052206897 0.6117637 1.239289
## 17 0.055620690 0.6145159 1.215949
## 18 0.059034483 0.6162015 1.204667
## 19 0.062448276 0.6168514 1.201099
## 20 0.065862069 0.6173366 1.197765
## 21 0.069275862 0.6178000 1.194924
## 22 0.072689655 0.6177435 1.194566
## 23 0.076103448 0.6175000 1.194560
## 24 0.079517241 0.6169604 1.195489
## 25 0.082931034 0.6163808 1.196755
## 26 0.086344828 0.6156689 1.198288
## 27 0.089758621 0.6148301 1.200071
## 28 0.093172414 0.6141295 1.201688
## 29 0.096586207 0.6132244 1.203602
## 30 0.100000000 0.6121841 1.205564
```

```r
# Plot tuning results
plot(lasso_model, main = "LASSO Model Tuning")
```




```r
# Step 6: Evaluate on test set
test_predictions <- predict(lasso_model, test_x_processed)
test_R2 <- cor(test_predictions, test_y)^2
test_RMSE <- sqrt(mean((test_y - test_predictions)^2))

cat("\nTest Set Performance:\n")
```

```
##
## Test Set Performance:
```

```r
cat("  R²:", round(test_R2, 4), "\n")
```

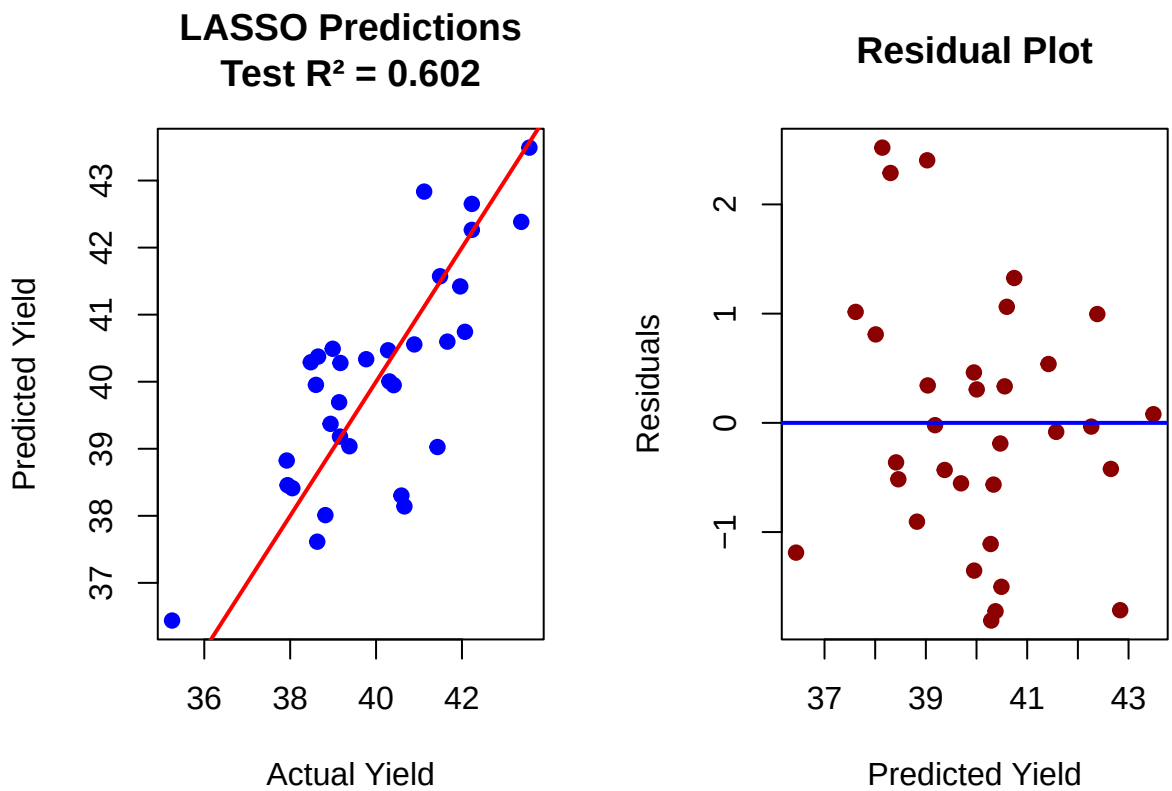
```
##   R²: 0.6025
```

```r
cat("  RMSE:", round(test_RMSE, 4), "\n")
```

```
##   RMSE: 1.1447
```

```r
# Step 7: Visualize predictions vs actual
par(mfrow = c(1, 2))
plot(test_y, test_predictions,
     xlab = "Actual Yield",
     ylab = "Predicted Yield",
     main = paste("LASSO Predictions\nTest R² =", round(test_R2, 3)),
     pch = 19, col = "blue")
abline(0, 1, col = "red", lwd = 2)

# Residual plot
residuals <- test_y - test_predictions
plot(test_predictions, residuals,
```

```
     xlab = "Predicted Yield",
     ylab = "Residuals",
     main = "Residual Plot",
     pch = 19, col = "darkred")
abline(h = 0, col = "blue", lwd = 2)
```



Summary of Results:

The LASSO regression model was tuned using 10-fold cross-validation on the training set (n=144). The optimal regularization parameter was lambda = 0.0761, achieving a cross-validated R^2 of 0.6178 (RMSE = 1.1949).

(d) Predict the response for the test set. What is the value of the performance metric and how does this compare with there sampled performance metric on the training set?

The test set achieved an R^2 of 0.6025 (RMSE = 1.1447), which is consistent with the cross-validated training performance of $R^2 = 0.6178$ (RMSE = 1.1949). The test set R^2 is only 0.0153 (approximately 2.5%) lower than the cross-validated estimate, and the test RMSE is actually slightly better (1.1447 vs 1.1949). This close agreement indicates that the LASSO model generalizes very well to unseen data with minimal overfitting. The small difference between resampled and test performance suggests that the 10-fold cross-validation provided an accurate estimate of model performance.

(e) Which predictors are most important in the model you have trained? Do either the biological or process predictors dominate the list?

```
# Extract coefficients from the LASSO model
coef_matrix <- coef(lasso_model$finalModel, s = lasso_model$bestTune$lambda)

# Convert to matrix and remove intercept
coef_vector <- as.matrix(coef_matrix)[-1, 1]  # Remove intercept
```

```r
# Get non-zero coefficients (important predictors)
important_vars <- coef_vector[coef_vector != 0]
important_df <- data.frame(
  Predictor = names(important_vars),
  Coefficient = important_vars,
  Abs_Coefficient = abs(important_vars)
)

# Sort by absolute coefficient magnitude (most important first)
important_df <- important_df[order(-important_df$Abs_Coefficient), ]

# Display results
cat("Number of predictors with non-zero coefficients:", length(important_vars), "\n")
```

```
## Number of predictors with non-zero coefficients: 20
```

```r
cat("Total predictors in model:", length(coef_vector), "\n")
```

```
## Total predictors in model: 57
```

```r
cat("Percentage selected:", round(100 * length(important_vars) / length(coef_vector), 1), "%\n\n")
```

```
## Percentage selected: 35.1 %
```

```r
cat("Top 15 most important predictors:\n")
```

```
## Top 15 most important predictors:
```

```r
print(head(important_df, 15))
```

```
##                                   Predictor Coefficient Abs_Coefficient
## ManufacturingProcess32 ManufacturingProcess32  0.99522517      0.99522517
## ManufacturingProcess09 ManufacturingProcess09  0.53440883      0.53440883
## ManufacturingProcess17 ManufacturingProcess17 -0.25291334      0.25291334
## ManufacturingProcess39 ManufacturingProcess39  0.12938830      0.12938830
## ManufacturingProcess07 ManufacturingProcess07 -0.11153988      0.11153988
## BiologicalMaterial05     BiologicalMaterial05  0.10456719      0.10456719
## ManufacturingProcess01 ManufacturingProcess01  0.09222140      0.09222140
## ManufacturingProcess06 ManufacturingProcess06  0.09091383      0.09091383
## ManufacturingProcess37 ManufacturingProcess37 -0.08667207      0.08667207
## ManufacturingProcess45 ManufacturingProcess45  0.07744441      0.07744441
## ManufacturingProcess13 ManufacturingProcess13 -0.05741043      0.05741043
## ManufacturingProcess24 ManufacturingProcess24 -0.04525195      0.04525195
## BiologicalMaterial06     BiologicalMaterial06  0.04118283      0.04118283
## BiologicalMaterial03     BiologicalMaterial03  0.03900646      0.03900646
## ManufacturingProcess35 ManufacturingProcess35 -0.03251921      0.03251921
```

```r
# Separate biological and process predictors
biological_vars <- important_df[grep("BiologicalMaterial", important_df$Predictor), ]
process_vars <- important_df[grep("ManufacturingProcess", important_df$Predictor), ]

cat("\n--- Summary by Predictor Type ---\n")
```

```
##
## --- Summary by Predictor Type ---
```

```r
cat("Biological predictors selected:", nrow(biological_vars), "out of 12\n")
```

```
## Biological predictors selected: 4 out of 12
```

```r
cat("Process predictors selected:", nrow(process_vars), "out of 45\n")
```

```
## Process predictors selected: 16 out of 45
```

```r
# Calculate total importance (sum of absolute coefficients)
bio_importance <- sum(abs(biological_vars$Coefficient))
process_importance <- sum(abs(process_vars$Coefficient))
total_importance <- bio_importance + process_importance

cat("\n--- Relative Importance ---\n")
```

```
##
## --- Relative Importance ---
```

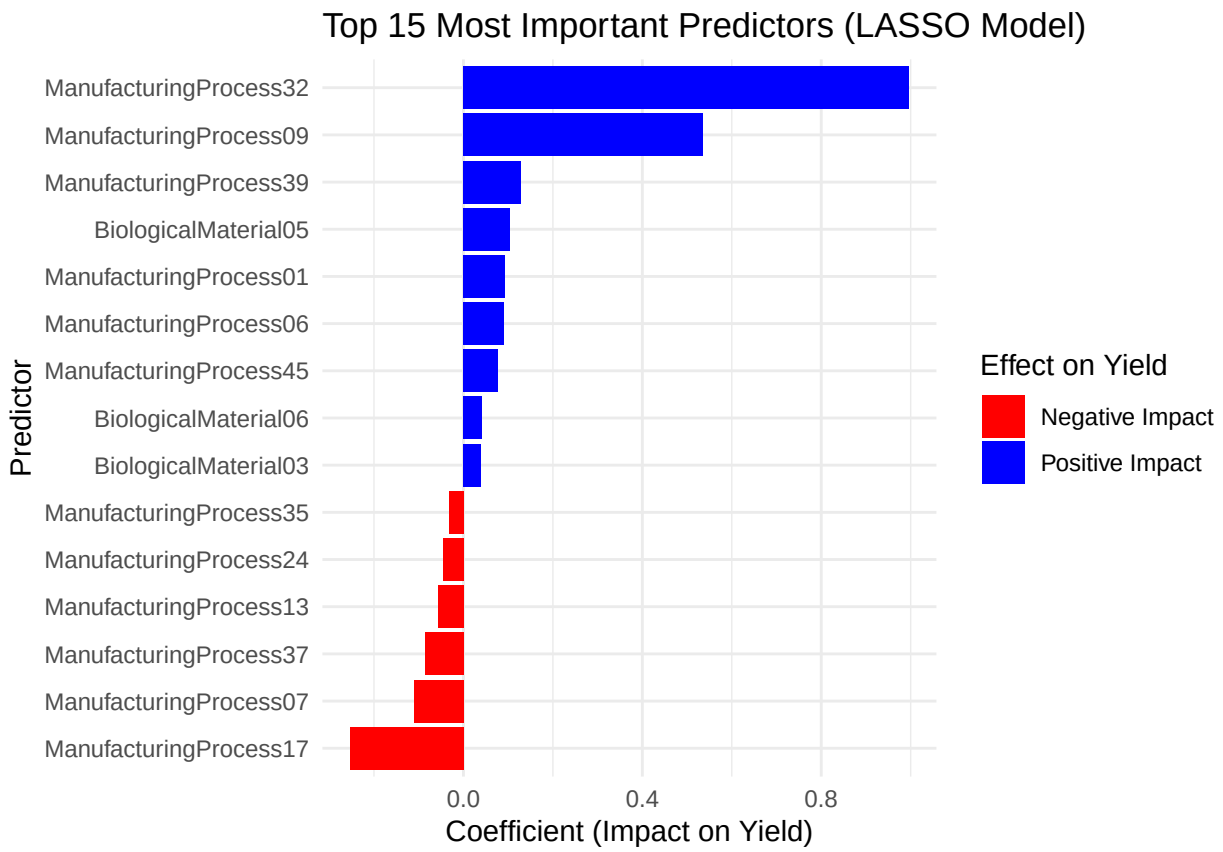
```r
cat("Biological predictors importance:", round(100 * bio_importance / total_importance, 1), "%\n")
```

```
## Biological predictors importance: 7.3 %
```

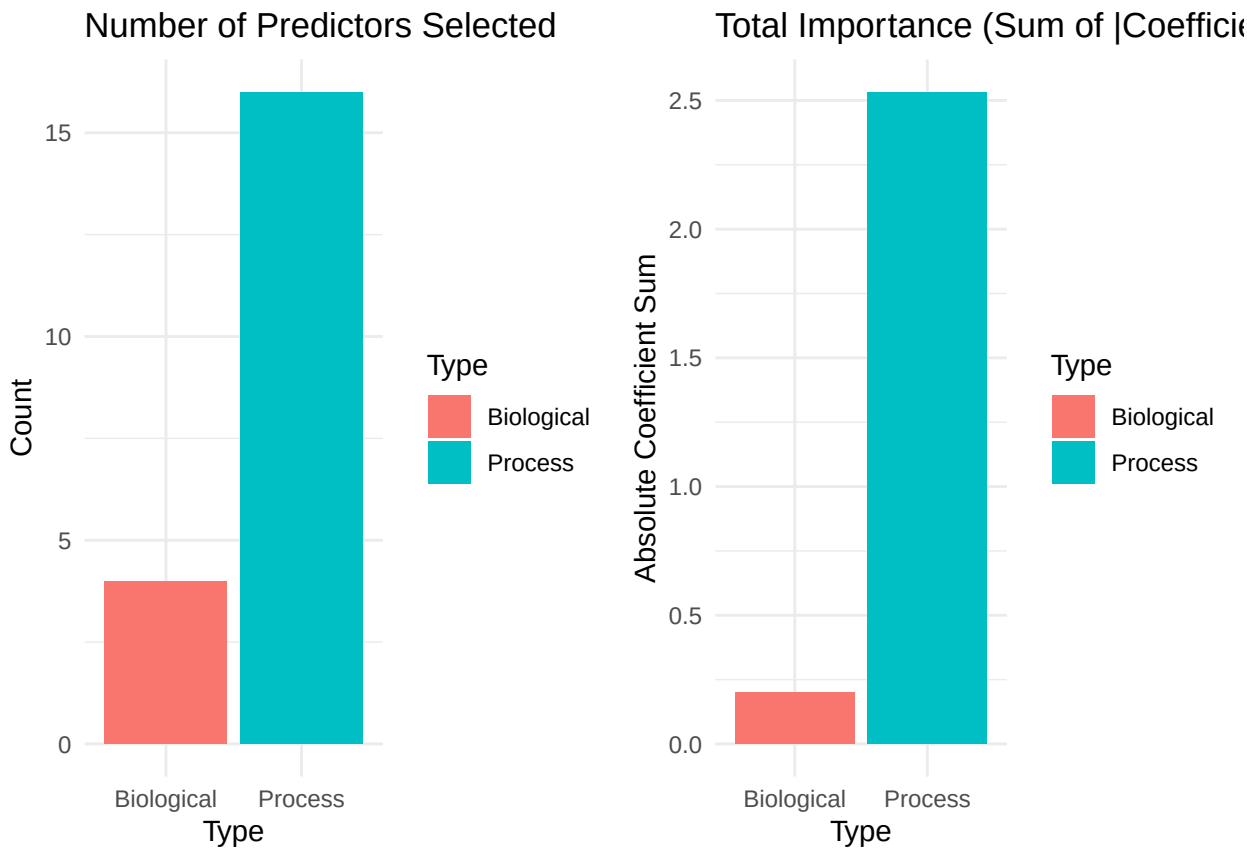
```r
cat("Process predictors importance:", round(100 * process_importance / total_importance, 1), "%\n")
```

```
## Process predictors importance: 92.7 %
```

```r
# Plot top 15 predictors
top15 <- head(important_df, 15)
ggplot(top15, aes(x = reorder(Predictor, Coefficient), y = Coefficient, fill = Coefficient > 0)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(title = "Top 15 Most Important Predictors (LASSO Model)",
       x = "Predictor", y = "Coefficient (Impact on Yield)") +
  theme_minimal() +
  scale_fill_manual(values = c("red", "blue"),
                    labels = c("Negative Impact", "Positive Impact"),
                    name = "Effect on Yield")
```




```r
# Compare biological vs process predictor importance
importance_comparison <- data.frame(
  Type = c("Biological", "Process"),
  Count = c(nrow(biological_vars), nrow(process_vars)),
  Importance = c(bio_importance, process_importance)
)

# Plot comparison
p1 <- ggplot(importance_comparison, aes(x = Type, y = Count, fill = Type)) +
  geom_bar(stat = "identity") +
  labs(title = "Number of Predictors Selected", y = "Count") +
  theme_minimal()

p2 <- ggplot(importance_comparison, aes(x = Type, y = Importance, fill = Type)) +
  geom_bar(stat = "identity") +
  labs(title = "Total Importance (Sum of |Coefficients|)", y = "Absolute Coefficient Sum") +
  theme_minimal()

gridExtra::grid.arrange(p1, p2, ncol = 2)
```



Summary:

The LASSO model selected 20 out of 57 total predictors (35.1%), with process predictors dominating the list. The most important predictor by far is ManufacturingProcess32 (coefficient = 0.995), followed by ManufacturingProcess09 (0.534) and ManufacturingProcess17 (-0.253).

1. Process predictors dominate with:

- 16 out of 20 selected predictors (80%) being process variables
- 92.7% of total importance (sum of absolute coefficients)

2. Biological predictors play a minor role:

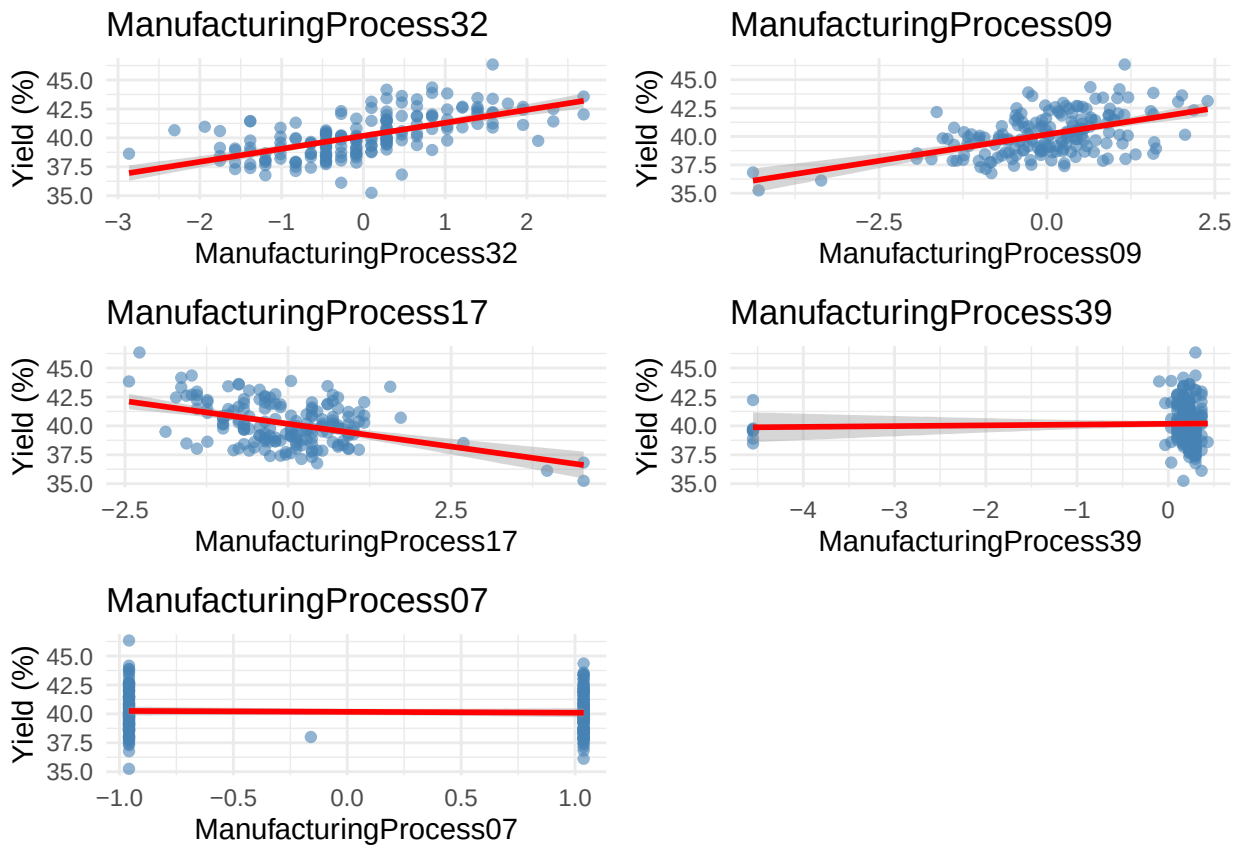
- Only 4 out of 12 biological variables selected
- Just 7.3% of total importance

Key Finding: The strong dominance of controllable process predictors is excellent news for manufacturing optimization. The top positive coefficients (ManufacturingProcess32, ManufacturingProcess09, ManufacturingProcess39) identify parameters that should be increased to improve yield, while negative coefficients (ManufacturingProcess17, ManufacturingProcess07) indicate parameters that should be decreased.

(f) Explore the relationships between each of the top predictors and the response. How could this information be helpful in improving yield in future runs of the manufacturing process?

```
# Explore relationships between top predictors and yield
top_predictors <- important_df$Predictor[1:5]

# Create scatter plots
plot_list <- list()
for(i in 1:length(top_predictors)) {
  predictor_values <- predictors_imputed[[top_predictors[i]]]
```

```r
  plot_list[[i]] <- ggplot(data.frame(x = predictor_values, y = yield),
                           aes(x = x, y = y)) +
    geom_point(alpha = 0.6, color = "steelblue") +
    geom_smooth(method = "lm", se = TRUE, color = "red") +
    labs(title = top_predictors[i], x = top_predictors[i], y = "Yield (%)") +
    theme_minimal()
}
grid.arrange(grobs = plot_list, ncol = 2, nrow = 3)
```




```r
# Summary statistics for top predictors
cat("\n========== RELATIONSHIP SUMMARY ==========\n")
```

```
##
## ========== RELATIONSHIP SUMMARY ==========
```

```r
for(i in 1:length(top_predictors)) {
  predictor_values <- predictors_imputed[[top_predictors[i]]]
  lm_model <- lm(yield ~ predictor_values)
  coef_val <- coef(lm_model)[2]
  p_val <- summary(lm_model)$coefficients[2, 4]
  r2 <- summary(lm_model)$r.squared

  cat(sprintf("\n%s: coef = %.3f, R² = %.3f, p = %.4f %s",
              top_predictors[i], coef_val, r2, p_val,
              if(p_val < 0.05) "(SIGNIFICANT)" else ""))
  cat(sprintf("\n  → %s this variable to improve yield",
              ifelse(coef_val > 0, "INCREASE", "DECREASE")))
}
```

```
##
## ManufacturingProcess32: coef = 1.123, R² = 0.370, p = 0.0000 (SIGNIFICANT)
##   → INCREASE this variable to improve yield
## ManufacturingProcess09: coef = 0.929, R² = 0.253, p = 0.0000 (SIGNIFICANT)
##   → INCREASE this variable to improve yield
## ManufacturingProcess17: coef = -0.786, R² = 0.181, p = 0.0000 (SIGNIFICANT)
##   → DECREASE this variable to improve yield
## ManufacturingProcess39: coef = 0.066, R² = 0.001, p = 0.6365
##   → INCREASE this variable to improve yield
## ManufacturingProcess07: coef = -0.078, R² = 0.002, p = 0.5793
##   → DECREASE this variable to improve yield
```

Result Summary

Statistically significant predictors ($p < 0.05$):

- ManufacturingProcess32 has the strongest positive relationship ($R^2 = 37.0\%$). Increasing this variable will likely produce the largest yield gains.
- ManufacturingProcess09 shows a moderate positive relationship ($R^2 = 25.3\%$). Increasing this variable also improves yield.
- ManufacturingProcess17 exhibits a negative relationship ($R^2 = 18.1\%$). Decreasing this variable increases yield.

Actionable recommendations:

- Primary focus: Increase ManufacturingProcess32 (largest impact)
- Secondary focus: Increase ManufacturingProcess09 and decrease ManufacturingProcess17
- Ignore: Process39 and Process07 (not statistically significant)
- Expected outcome: Focusing on the three significant predictors (Process32, Process09, Process17) provides a clear, data-driven path to optimize the manufacturing process and improve yield in future runs.